

TRIBHUWAN UNIVERSITY
INSTITUTE OF ENGINEERING
KATHMANDU ENGINEERING COLLEGE
KALIMATI, KATHMANDU



Minor Project
Mid-term Defense Report
On
INTELLIGENT MENTAL HEALTH MONITORING SYSTEM
BASED ON ML
[Code Number: ENCT354]

BY

Abhipsa Nepal	[KAT080BCT006]
Abisha Dhungel	[KAT080BCT007]
Anushesh Ghimire	[KAT080BCT016]
Ayush Gyawali	[KAT080BCT023]

TO
DEPARTMENT OF COMPUTER ENGINEERING
KATHMANDU, NEPAL

Asar, 2083

ACKNOWLEDGEMENT

We would like to take this opportunity to thank all those who supported us in the development of the Intelligent Mental Health Monitoring System Based on Machine Learning.

We would like to take this opportunity to appreciate our Project Coordinator, **Er. Krista Byanju**, for her constant support and motivation. We would also like to acknowledge the support provided by **Er. Sudeep Shakya**, the Head of Department of Computer Engineering and **Er. Kunjan Amatya**, the Deputy Head of Department of Computer Engineering.

It is with a sense of heartfelt appreciation that we acknowledge the entire faculty of the **Kathmandu Engineering College** for their teaching, advice, and the strong academic foundation they provided.

We'd like to thank all of you for making this project possible.

ABSTRACT

Mental health issues have become a major concern worldwide, and traditional assessment methods often delay timely identification and support. To address this challenge, this project proposes an Intelligent Mental Health Monitoring System based on Machine Learning that enables continuous and proactive mental health monitoring through data analysis. The system allows users to securely record daily journal entries, moods, stress levels, sleep patterns, and personal reflections. User-provided textual data is analyzed using Natural Language Processing techniques to identify emotional patterns, while a supervised machine learning algorithm, specifically Random Forest, is used to evaluate and classify mental health states. An intelligent chatbot module is also integrated to encourage regular engagement by providing journaling prompts, supportive interactions, and self-care guidance. The system presents analyzed results through clear visual summaries, helping users better understand their emotional trends over time and even recommends mindfulness exercises and suggests to a clinical personnel if stress is high preventing future risk. With a strong focus on accessibility, privacy, and simplicity, the proposed system aims to support mental health awareness and self-reflection while complementing, rather than replacing, professional medical care.

Keywords: *Mental Health Monitoring, Machine Learning, Natural Language Processing, Random Forest*

TABLE OF CONTENTS

ACKNOWLEDGEMENT.....	ii
ABSTRACT	iii
TABLE OF CONTENTS	iv
LIST OF FIGURES.....	vii
LIST OF TABLES.....	viii
LIST OF ABBREVIATIONS.....	ix
CHAPTER 1: INTRODUCTION	1
1.1 BACKGROUND	1
1.1.1 Natural Language Processing (NLP).....	1
1.1.2 Sentiment Analysis.....	2
1.2 PROBLEM STATEMENT	2
1.3 OBJECTIVES.....	3
1.4 SCOPE AND APPLICATIONS.....	3
1.4.1 SCOPE.....	3
1.4.2 APPLICATIONS:	3
CHAPTER 2: LITERATURE REVIEW.....	5
2.1 EXISTING SYSTEMS	6
2.2 LIMITATIONS OF PREVIOUS SYSTEMS.....	7
2.3 SOLUTIONS PROPOSED BY OUR SYSTEM.....	7
CHAPTER 3: METHODOLOGY	8
3.1 PROCESS MODEL	8
3.2 BLOCK DIAGRAM.....	11

3.3 ALGORITHM	14
3.3.1 SENTIMENT ANALAYSIS USING VADER	15
3.3.2 RANDOM FOREST CLASSIFICATION	15
3.4 FLOWCHART	16
3.5. UML DIAGRAM	17
3.5.1 USE CASE DIAGRAM	17
3.5.2 ACTIVITY DIAGRAM	18
3.6 DATA FLOW DIAGRAMS	19
3.6.1 DFD LEVEL 0.....	19
3.6.2 DFD LEVEL 1.....	20
3.7 TOOLS USED.....	21
3.7.1 HTML (Hyper Text Markup Language):	21
3.7.2 CSS (Cascading Style Sheets):	21
3.7.3 Numpy and Pandas:.....	21
3.7.4 JavaScript:	21
3.7.5 Visual Studio Code (VS Code):.....	21
3.7.6 Python Compiler:	22
3.7.7 NLTK (Natural Language Toolkit):.....	22
3.7.8 Scikit-learn:.....	22
3.7.9 VADER (Valence Aware Dictionary and Sentiment Reasoner):	22
3.7.10 Jupyter Notebook:	22
3.7.11 MongoDB:	22
3.7.12 Pymongo	23
3.7.13 Git and GitHub.....	23
3.7.14 TF-IDF (Term Frequency-Inverse Document Frequency) Vectorizer.....	23

3.7.15 Flask Server:	23
3.7.16 Datasets:.....	24
3.7.17: APIS USED:	24
3.8 VERIFICATION AND VALIDATION	26
3.8.1 VERIFICATION.....	26
3.8.2 VALIDATION.....	27
CHAPTER 4: EPILOGUE.....	28
4.1 TASKS COMPLETED	28
4.2. TASKS REMAINING	28
REFERENCES	29
BIBLIOGRAPHY.....	30
SCREENSHOTS	31

LIST OF FIGURES

Figure 3.1:	Incremental Process Model	8
Figure 3.2:	Block Diagram of the System.....	11
Figure 3.4:	System Flowchart	16
Figure 3.5.1:	Use Case Diagram of the System	17
Figure 3. 5.2:	Activity Diagram of the System	18
Figure 3. 6.1:	Data Flow Diagram (Level 0) for Mental Health Monitoring System	19
Figure 3. 6.1:	Data Flow Diagram (Level 1) for Mental Health Monitoring System	20

LIST OF TABLES

Table 1: Backend APIS.....	25
----------------------------	----

LIST OF ABBREVIATIONS

AI:	Artificial Intelligence
CSS:	Cascading Style Sheets
HTML:	Hyper Text Markup Language
ML:	Machine Learning
NLP:	Natural Language Processing
NLTK:	Natural Language Toolkit
TF-IDF:	Term Frequency-Inverse Document Frequency
VADER:	Valence Aware Dictionary and Sentiment Reasoner
VS Code:	Virtual Studio Code

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND

Mental health is an essential aspect of overall well-being that affects how individuals think, feel, and behave in their daily lives. In recent years, mental health issues have become increasingly common due to factors such as academic pressure, work-related stress, social isolation, financial challenges, and lifestyle changes driven by rapid technological advancement. Conditions such as stress, anxiety, and emotional exhaustion often develop gradually and may remain unnoticed until they significantly impact an individual's quality of life and productivity.

Traditional mental health assessment methods primarily rely on self-reported questionnaires and periodic clinical evaluations. Although these methods can provide useful insights, they often fail to capture day-to-day emotional changes and early indicators of psychological distress. At the same time, digital platforms have become a major medium through which individuals express their thoughts and emotions. This creates an opportunity to utilize Artificial Intelligence (AI) and Machine Learning (ML) techniques for continuous and personalized mental health monitoring.

1.1.1 Natural Language Processing (NLP)

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, interpret, analyze, and generate human language in a meaningful way. It combines concepts from computer science, linguistics, and machine learning to allow systems to process unstructured textual data and extract useful information from it. NLP performs various tasks such as text preprocessing, tokenization, stop-word removal, stemming, lemmatization, keyword extraction, language understanding, and context analysis. These processes help transform raw text into a structured format that can be analyzed by computational models.

NLP is widely used in applications such as chatbots, virtual assistants, machine translation, speech recognition, text summarization spam filtering, and sentiment

analysis. In our proposed system NLP plays a crucial role in processing users' journal entries and written expressions. By analyzing the words, phrases, and context of the text, NLP enables the system to understand emotional cues and prepare the data for further analysis.

1.1.2 Sentiment Analysis

Sentiment Analysis is a specialized application of NLP that focuses on identifying, extracting, and evaluating the emotional tone expressed in textual data. It aims to determine whether a piece of text conveys a positive, negative, or neutral sentiment and may also measure the strength or intensity of the expressed emotion. Sentiment analysis uses computational techniques, linguistic rules, and machine learning approaches to analyze words, phrases, and sentence structures that indicate emotions and opinions.

Various sentiment analysis methods exist, including lexicon-based approaches, machine learning-based approaches, and hybrid techniques. In this project, sentiment analysis is performed using VADER (Valence Aware Dictionary and Sentiment Reasoner), a lexicon- and rule-based sentiment analysis tool specifically designed for analyzing sentiments in textual content. VADER assigns sentiment scores based on the emotional polarity of words and expressions and produces metrics such as positive, negative, neutral, and compound scores. These scores help quantify the emotional state reflected in a user's journal entry.

1.2 PROBLEM STATEMENT

Stress and other mental health concerns often begin at an early stage and gradually intensify if left unnoticed. Many individuals fail to recognize the initial sign of emotional distress until the condition begins to significantly affect their well-being, relationships, and daily activities. Existing mental health assessment approaches primarily rely on periodic clinical evaluations and self-reported questionnaires. These methods provide only limited insights into an individual's mental state at a particular point in time and may fail to capture continuous emotional fluctuations. As a result, subtle changes in mood and early indicators of stress often remain unnoticed.

In addition, social stigma, lack of awareness, time constraints, and limited access to mental health services discourage many individuals from seeking professional support when needed. Although people frequently express their thoughts and emotions through written journals and digital text, this information is often not systematically assessed for emotional patterns.

1.3 OBJECTIVES

To develop an intelligent mental health monitoring system that analyzes user-generated textual data using Natural Language Processing (NLP) and Machine Learning (ML) techniques to detect stress levels and provide meaningful insights and recommendations.

1.4 SCOPE AND APPLICATIONS

1.4.1 SCOPE

The Intelligent Mental Health Monitoring System is intended to support users in monitoring their personal mental health in terms of mood logging, analyzing stress, writing a personal journal, and analyzing moods. For this purpose, not only will the system support interactive operations with a chatbot system, but it will also ensure privacy in user data. This system will be useful in raising awareness and will not support overt medical services.

1.4.2 APPLICATIONS:

1. Personal Mental Health Tracking

Persons can utilize the system to monitor moods, stress levels on a daily basis.

2. Self-Reflection and Journaling

Journaling also enables a user to write down their ideas, thus enhancing emotional clarity.

3. Mental Health Awareness

The system can be used as an awareness tool for the user to be able to understand emotional patterns and the significance of mental wellness.

4. Student Support

Students can employ the system to cope with school stress, anxiety, and emotional issues. Workplace Well-Being Support Employees are also able to utilize the system for work-life balance and stress level checks. It can be utilized for educational use to demonstrate intelligent systems, data analysis, and human computer interaction.

CHAPTER 2: LITERATURE REVIEW

Recent research has shown that Natural Language Processing (NLP) and Machine Learning (ML) techniques can be effectively used for detecting stress from textual data. Saha et al. [1] proposed a method for stress detection using social media content, where machine learning algorithms were applied to analyze linguistic patterns. Their study demonstrated that textual features can be used to identify stress-related expressions. However, the approach mainly focused on data analysis and lacked a personalized monitoring system for continuous user interaction.

Lin et al. [2] introduced a deep learning-based approach for detecting psychological stress at the user level from social media data. The study emphasized analyzing user behavior over time rather than relying on single instances of text. While their approach achieved high accuracy, it relied on complex neural network models, which may reduce interpretability and increase computational complexity.

Shah et al. [3] proposed a machine learning-based mental health assistant to help users manage stress and anxiety through a mobile application. The system uses an expert-designed questionnaire to assess users' stress levels and employs several machine learning algorithms, selecting Random Forest as the final model due to its superior predictive performance.

.Zhang et al. [4] applied machine learning algorithms, including Random Forest and Support Vector Machines, for stress detection from text data. Their results showed that ensemble learning methods can improve classification performance. However, the study primarily focused on model accuracy and did not incorporate user interaction or visualization features for better usability.

Khan et al. [5] presented a comprehensive review of machine learning techniques used for mental stress detection. The study analyzed various algorithms and highlighted the effectiveness of classification models such as Random Forest. It also emphasized the need for systems that are not only accurate but also user-friendly and capable of continuous monitoring, which forms the basis for the proposed system.

2.1 EXISTING SYSTEMS

Mental health monitoring has become increasingly important due to rising stress, anxiety, and emotional challenges in daily life. Digital tools have emerged to help individuals track their moods, reflect on emotions, and receive mental health support conveniently. These tools aim to empower users to better understand their mental state and adopt practices that improve well-being. Some of the existing applications are:

1. Mood Tracking Applications

Examples: Daylio, Moodfit

Functionality: Allow users to log moods, track emotional patterns over time, and gain insights into triggers or trends.

2. Journaling Platforms

Examples: Journey, Reflectly

Functionality: Provide a digital diary for users to reflect on daily experiences, record thoughts, and practice self-expression.

3. Chatbot-Based Support Systems

Examples: Wysa, Woebot

Functionality: Offer AI-driven conversations to provide emotional support, coping techniques, and guided exercises for mental well-being.

4. Well-Being Platforms

Examples: Headspace, Calm

Functionality: Focus on meditation, mindfulness, stress reduction, and overall mental wellness through structured programs and audio-guided sessions.

2.2 LIMITATIONS OF PREVIOUS SYSTEMS

Currently, the mental health monitoring systems that exist, such as mood-tracking applications, journaling applications and mental well-being applications (such as Daylio, Moodfit, Wysa, Woebot, Headspace and Calm) are characterized by numerous drawbacks. For example, the majority of these applications rely on manual mood tracking and rule-based statistical analysis, lacking intelligent interpretation of user data. Moreover, these applications provide minimal analytics insights and feedback, making self-awareness and mental-state-tracking inefficient. Further, for the majority of the applications, it is essential for the users to engage regularly with the applications, and as such, some issues may arise with regards to confidentiality of information provided to the applications.

2.3 SOLUTIONS PROPOSED BY OUR SYSTEM

- The Intelligent Mental Health Monitoring System design intends to cover the shortcomings of similar systems by offering a user-friendly system for mental health monitoring.
- The proposed system integrates Machine Learning models for pattern recognition and NLP techniques for text sentiment analysis allowing for intelligent assessment of user's mental health conditions.
- Users can monitor their moods, stress levels, using the system's journaling feature. They can also view basic analysis results for their mental health, such as a summary of changes in emotions over a specific period of time.
- Furthermore, the Intelligent Mental Health Monitoring System includes a chatbot utility for interactive support, journaling, for mental health awareness. As a self-help system for mental health awareness, this design focuses on simplicity, usability, and the importance of mental health care, rather than offering alternative medical assistance.

CHAPTER 3: METHODOLOGY

3.1 PROCESS MODEL

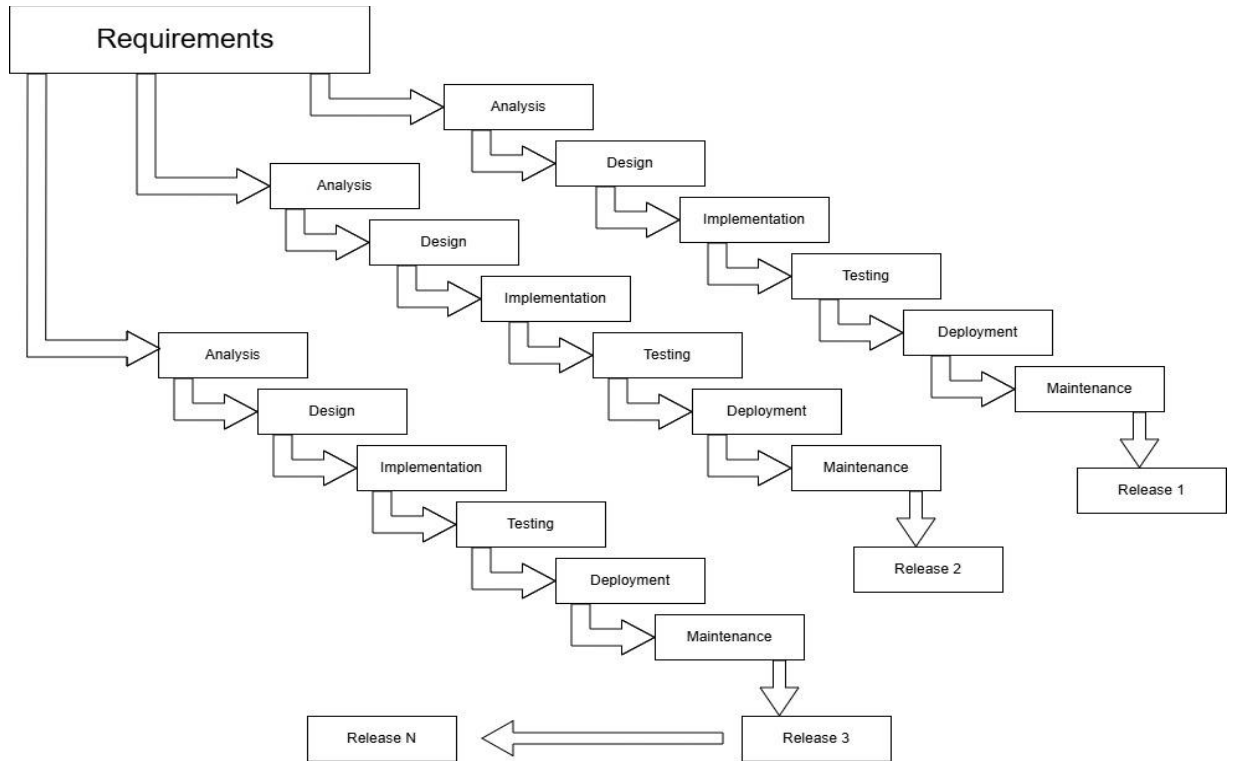


Figure 3.1: Incremental Process Model

For this Project, we are using the Incremental Model.

The Incremental Model is a software development approach where the system is designed, implemented, and tested in small pieces (increments), and each increment adds new functionality to the previous one.

Working of Incremental model

Analysis:

In this step, the requirements for the current increment are gathered and studied carefully. Only the features planned for this particular increment are considered, which makes the development focused and manageable.

Design:

During design, the system architecture and detailed design for the increment are created. This includes planning how the new features will fit with previously developed increments, defining module structures, interfaces, and data flow.

Implementation:

The implementation step involves coding the increment based on the design specifications. Developers write the program modules, integrate necessary components, and prepare the increment to be tested.

Testing:

Once implemented, the increment undergoes testing to verify that it meets the requirements and works correctly both individually and with previous increments.

Deployment:

After testing, the increment is deployed or integrated into the existing system. Users can begin using the new features immediately, which provides early value and helps gather feedback for the next increment.

Maintenance:

This step addresses any issues, bugs, or minor improvements identified after deployment. Maintenance ensures that each increment remains stable and functional, providing a solid foundation for subsequent increments without disrupting the existing system.

Release:

The increment is formally released to the user or client as a functional module. Each release adds new functionality to the system.

Increment 1: User Registration and Journal Writing (Front-End Development)

The first increment focuses on developing the user interface and basic system functionality. Users can create accounts, log in securely, and write daily journal entries through a user-friendly web interface. This increment establishes the foundation of the system by enabling users to record their thoughts and emotions digitally.

Increment 2: Chatbot Design

The second increment introduces a chatbot component to provide interactive communication within the system. The user interacts with the chatbot answer the specific questions provided by it.

Increment 3: Sentiment Analysis Implementation (Backend Development)

The third increment integrates Natural Language Processing (NLP) and sentiment analysis into the system. User journal entries are processed to identify emotional tone and sentiment. The sentiment analysis component evaluates the text and generates sentiment scores, enabling the system to understand the user's emotional state based on their written content.

Increment 4: Database Integration and Score Management

The fourth increment focuses on storing user information, journal entries, sentiment scores, and chatbot interactions in a database. The system also calculates average sentiment scores from multiple journal entries to support emotional trend analysis and maintain historical records for future reference.

Increment 5: Data Visualization and Mindfulness Suggestions

The fifth increment enhances the system by presenting sentiment data through graphical visualizations such as mood trend charts and reports. Based on the analyzed emotional state and trends, the system provides mindfulness suggestions and wellness recommendations to encourage positive mental health practices.

Final Integration and Deployment

After completing all increments, the individual modules are integrated into a single system. Comprehensive testing is performed to ensure proper communication between components, including user management, journaling, sentiment analysis, database storage, visualization, and recommendation features. The fully integrated system is then deployed for use.

3.2 BLOCK DIAGRAM

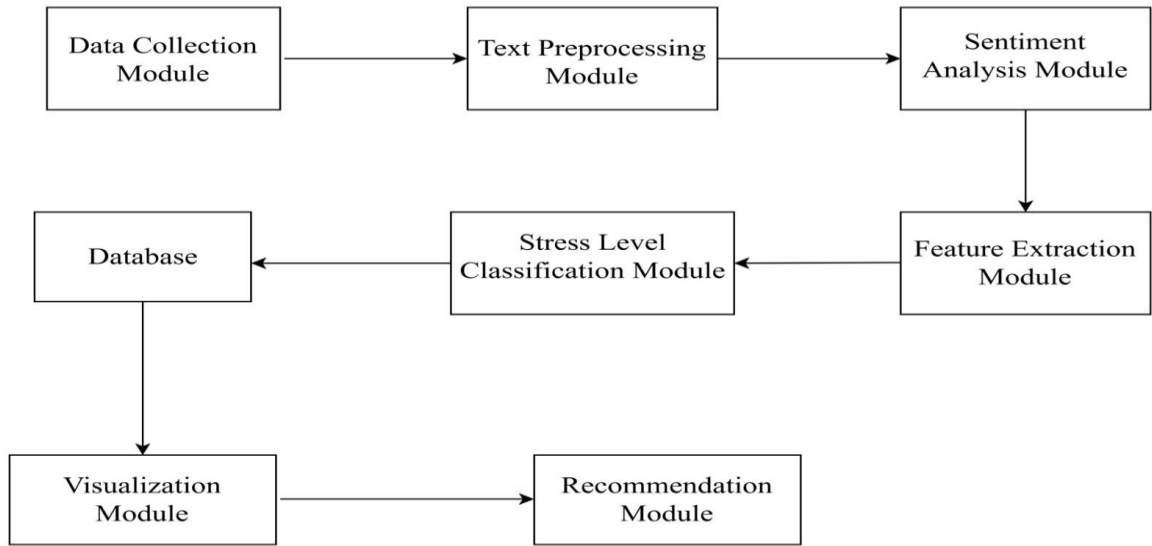


Figure 3.2: Block Diagram of the System

The proposed system is designed to monitor and analyze users' mental health by processing textual input using Natural Language Processing (NLP) and Machine Learning techniques. The overall workflow of the system is illustrated in the block diagram and is explained as follows

1. Data Collection Module

This module serves as the entry point of the system. It collects textual data from users through journaling interfaces and chatbot interactions. The data may include daily thoughts, emotional expressions, and personal reflections.

2. Text Preprocessing Module

The raw text collected from users is often unstructured and may contain noise such as punctuation, special characters, and inconsistent formatting. This module performs essential preprocessing steps including:

- Conversion of text to lowercase
- Removal of punctuation and special characters
- Removal of stop words (e.g., "the", "is", "and")

- Basic tokenization (splitting text into words)

These steps help in standardizing the input and reducing irrelevant information, thereby improving the efficiency and accuracy of the sentiment analysis and classification processes.

3. Sentiment Analysis Module

In this module, the cleaned text is analyzed using the VADER (Valence Aware Dictionary and Sentiment Reasoner) sentiment analysis tool. VADER is a lexicon rule-based NLP model that assigns sentiment scores to the input text. It produces values as:

- Positive score
- Negative score
- Neutral score
- Compound score (overall sentiment)

These scores quantitatively represent the emotional tone of the user's input. This module acts as a bridge between textual data and numerical analysis.

4. Feature Extraction Module

The output from the sentiment analysis module is transformed into structured numerical features suitable for machine learning models. These features may include:

- Sentiment scores (positive, negative, compound),
- Frequency of emotionally significant words
- Basic textual indicators (e.g., intensity of expressions)

Feature extraction is crucial because machine learning algorithms cannot process raw text directly; they require numerical representations of data.

5. Stress Level Classification Module

This is the core analytical component of the system. It uses both:

- Rule-based evaluation (based on predefined thresholds of sentiment scores)
- Machine Learning model (Random Forest)

The rule-based approach provides quick preliminary classification, while the Random Forest model improves accuracy by learning patterns from the data. The model analyzes the extracted features and classifies the user's mental state into categories such as low, moderate, or high stress. This combination ensures both interpretability and robustness in predictions.

6. Database

The database acts as a central storage system. It stores raw user input, sentiment scores, extracted features, final stress classification results. Storing this data enables long-term tracking of user behavior and supports future analysis. It also allows retrieval of data for visualization and trend monitoring.

7. Visualization Module

This module retrieves stored data from the database and presents it in graphical formats such as charts and graphs. Examples include: Daily or weekly mood trends

Visualization helps users easily interpret their emotional patterns and understand changes in their mental state without needing technical knowledge.

8. Recommendation Module

Based on the analyzed stress levels, this module provides appropriate feedback to the user. Recommendations may include: Breathing and Meditation, Guidance for seeking professional help (if needed)

This module ensures that the system not only analyzes data but also provides meaningful support to the user.

3.3 ALGORITHM

Algorithm for Intelligent Mental Health Monitoring System

STEP 1: Start the system and initialize the database connection.

STEP 2: Collect user input through chatbot or journal entry.

STEP 3: Preprocess the text: convert to lowercase, remove stop words, and eliminate special characters.

STEP 4: Apply VADER sentiment analysis to the preprocessed text to compute daily sentiment scores (positive, negative, neutral, and compound).

STEP 5: Generate daily mood score from the sentiment scores.

STEP 6: Store the daily mood score and user data into database for future analysis.

STEP 7: Retrieve stored mood data and perform statistical feature extraction: compute weekly average, variance, and mood trends using rule-based statistical methods.

STEP 8: Classify stress level using Random Forest with the extracted features as input.

STEP 9: Display mood trends and stress levels using graphs and charts

STEP 10: Generate recommendations for mindfulness or professional support based on stress classification.

STEP 11: End the process.

3.3.1 SENTIMENT ANALYSIS USING VADER

Sentiment analysis is an NLP technique used to determine the emotional tone expressed in textual data. VADER is a lexicon- and rule-based sentiment analysis model specifically designed to analyze sentiments expressed in short texts.

VADER works by assigning sentiment polarity scores based on a predefined dictionary of words along with linguistic rules that consider context, intensity, and negation. It produces numerical sentiment scores that indicate whether the expressed sentiment is positive, negative, or neutral.

In this project, VADER is used to analyze the preprocessed user text and generate sentiment scores that represent the user's emotional state. These scores provide a quantitative measure of daily mood, which serves as an important feature for identifying stress patterns.

3.3.2 RANDOM FOREST CLASSIFICATION

Random Forest is a supervised machine learning algorithm that belongs to the ensemble learning category. It consists of multiple decision trees, each trained on a random subset of the dataset and features. The final classification result is obtained by combining the predictions of all trees using a majority voting mechanism.

Random Forest is known for its robustness, high accuracy, and ability to handle complex and non-linear data patterns. It also reduces over fitting by averaging the results of multiple decision trees rather than relying on a single model.

In the proposed system, the Random Forest algorithm is used to classify stress levels based on sentiment-related features extracted from user text. By learning patterns from historical data, the model predicts whether the user's stress level falls into predefined categories such as low, moderate, or high stress.

3.4 FLOWCHART



Figure 3.3: System Flowchart

3.5. UML DIAGRAM

3.5.1 USE CASE DIAGRAM

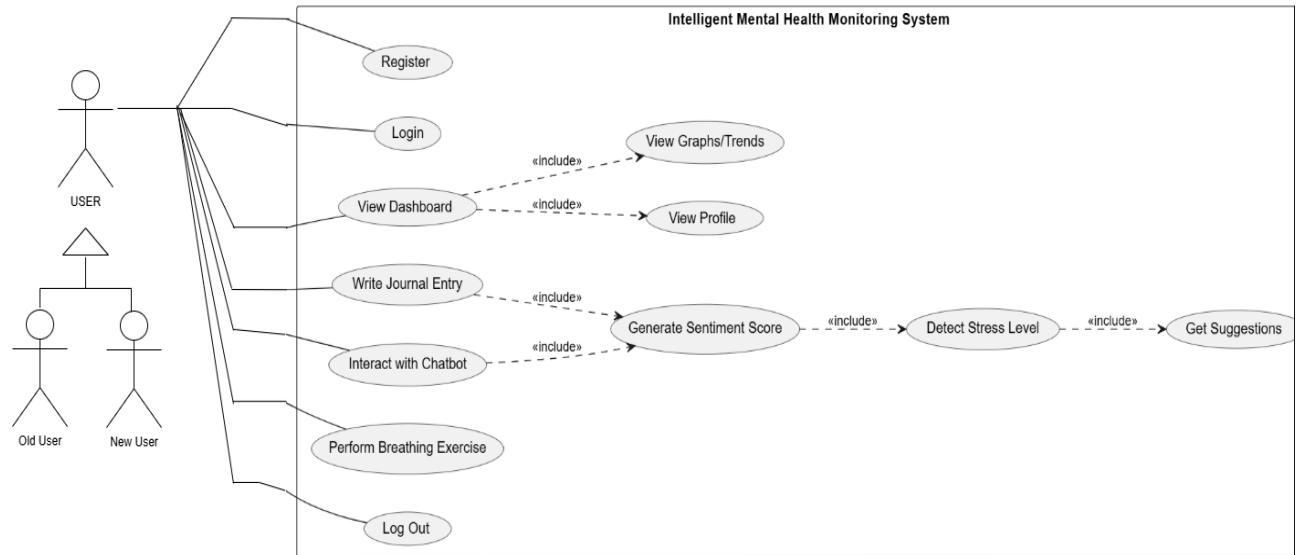


Figure 3.5.1: Use Case Diagram of the System

3.5.2 ACTIVITY DIAGRAM

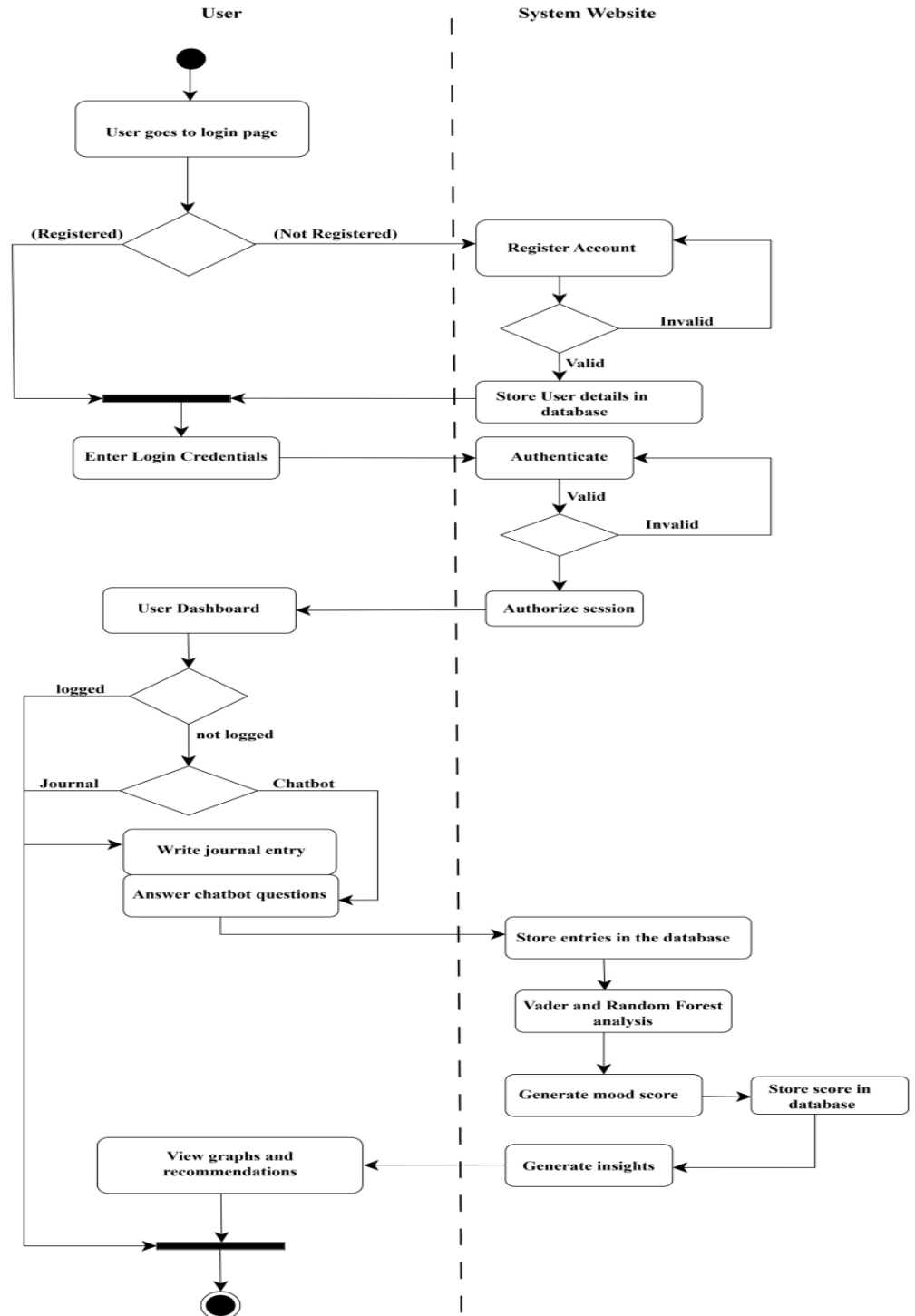


Figure 3. 4.2: Activity Diagram of the System

3.6 DATA FLOW DIAGRAMS

3.6.1 DFD LEVEL 0

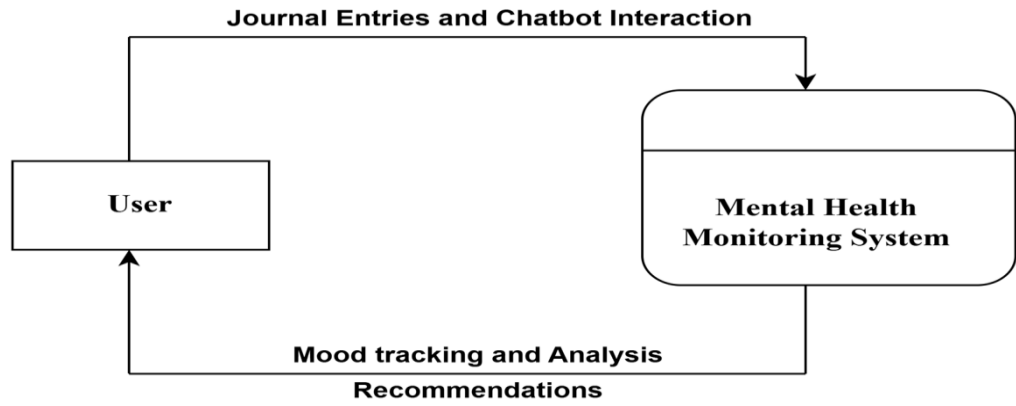


Figure 3. 5.1: Data Flow Diagram (Level 0) for Mental Health Monitoring System

3.6.2 DFD LEVEL 1

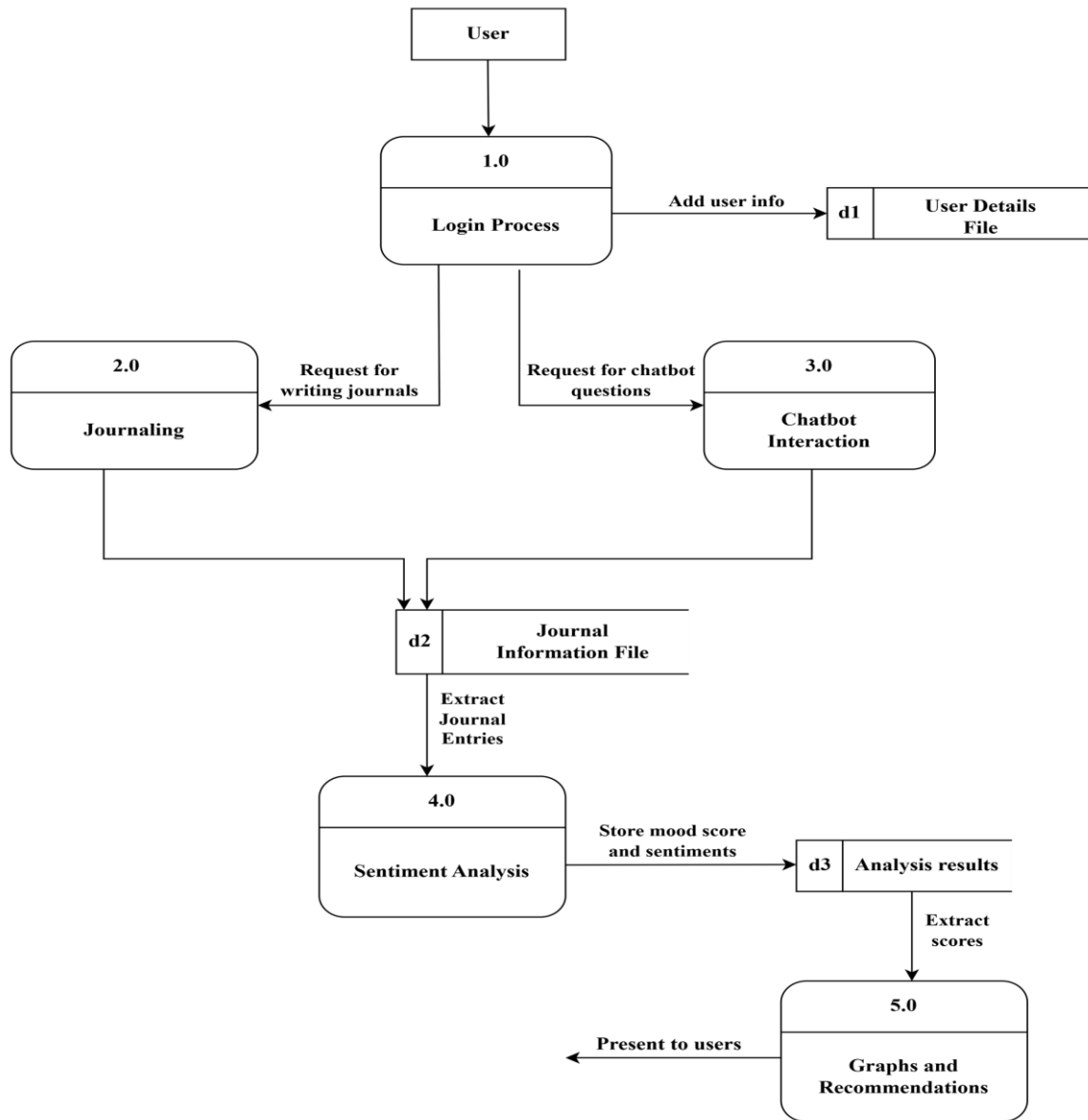


Figure 3. 6.1: Data Flow Diagram (Level 1) for Mental Health Monitoring System

3.7 TOOLS USED

3.7.1 HTML (Hyper Text Markup Language):

HTML is the standard markup language used to create the structure of web pages and applications. It allows the creation of elements such as headings, paragraphs, forms, tables, and links, forming the backbone of any web-based interface.

3.7.2 CSS (Cascading Style Sheets):

CSS is a style sheet language used to describe the presentation of HTML elements. It is used for styling the web pages and making it look presentable and nice. It controls the layout, colors, fonts, spacing, and visual design of web pages, helping create attractive and user-friendly interfaces.

3.7.3 Numpy and Pandas:

NumPy and Pandas are essential Python libraries used in data science and scientific computing. NumPy focuses on efficient numerical operations using multi-dimensional arrays and mathematical functions, making it fast and memory-efficient. Pandas, built on NumPy, is used for data manipulation and analysis, providing easy-to-use data structures like Series and Data Frames for handling structured data. Together, they simplify data processing, analysis, and preparation for machine learning applications.

3.7.4 JavaScript:

JavaScript is a high-level, interpreted programming language used to make web pages interactive. It enables dynamic content such as animations, form validation, interactive buttons, and responsive elements, making web applications more engaging for users.

3.7.5 Visual Studio Code (VS Code):

VS Code is a free, lightweight, and powerful source code editor. It supports multiple programming languages, debugging, version control, and extensions, making it a popular choice for software development and coding.

3.7.6 Python Compiler:

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. It is widely used in software development, artificial intelligence, and machine learning due to its extensive libraries and community support.

3.7.7 NLTK (Natural Language Toolkit):

NLTK is a Python library for natural language processing. It provides tools for text processing, tokenization, stop-word removal, stemming, parsing, and sentiment analysis, making it useful for analyzing and understanding human language data

3.7.8 Scikit-learn:

Scikit-learn is a Python library for machine learning. It is used in this project to implement the Random Forest algorithm for classifying the stress levels of users based on their mood scores.

3.7.9 VADER (Valence Aware Dictionary and Sentiment Reasoner):

VADER is a rule-based sentiment analysis algorithm included in NLTK. It analyzes the preprocessed text to generate numerical sentiment scores representing positive, negative, and neutral emotions, which are then used to calculate daily and weekly mood scores.

3.7.10 Jupyter Notebook:

Jupyter Notebook is an interactive computing environment that allows users to create and share documents containing live code, equations, visualizations, and narrative text. It is widely used in data science, machine learning, and research

3.7.11 MongoDB:

MongoDB is a NoSQL database designed for storing and managing unstructured or semi-structured data. It uses a flexible document-oriented data model, making it scalable, efficient, and suitable for handling large volumes of data.

3.7.12 Pymongo:

PyMongo is the official Python driver for MongoDB, a popular NoSQL database. It allows Python applications to connect to a MongoDB server, store data, query data, update documents, and perform database administration tasks. It works with local MongoDB servers and cloud-hosted MongoDB Atlas. It supports aggregation pipelines, indexes, transactions, and authentication and integrates well with Python frameworks like Flask, Django, and FastAPI

3.7.13 Git and GitHub:

Git is a distributed version control system used to track changes in source code during software development. It allows multiple developers to work on the same project simultaneously, manage different versions of files, and maintain a complete history of code changes, making collaboration and code management efficient and reliable.

GitHub is a cloud-based platform built on Git that enables developers to store, share, and collaborate on code repositories. It provides tools for version control, project management, issue tracking, and code review.

3.7.14 TF-IDF (Term Frequency-Inverse Document Frequency) Vectorizer

TF-IDF Vectorizer is a text preprocessing technique in Python used to convert textual data into numerical vectors. In Python, it is commonly implemented using the TfidfVectorizer class from the scikit-learn library. TF-IDF helps machine learning models understand and analyze text data effectively. It is widely used in text classification, sentiment analysis, spam detection, and information retrieval systems

3.7.15 Flask Server:

A Flask server is a web server created using the Flask framework in Python. Flask helps you build web applications and APIs that can receive requests from users and send responses back. The backend is implemented using Flask.

3.7.16 Datasets:

Kaggle Dataset "Sentiment Analysis for Mental Health Monitoring"

Scale & Structure: Contains 94,023 records with 3 columns (index, statement, status) . The dataset represents a large-scale collection of mental health-related text entries with categorical labels.

Content Type: Comprises naturally occurring mental health statements, primarily sourced from community forums or social platforms, capturing authentic emotional expressions and personal experiences related to mental health conditions.

Labeling System: Uses categorical status labels (Anxiety, Depression, etc.) to classify statements, enabling supervised learning applications for mental health sentiment classification and condition detection tasks.

Text Characteristics: Features unstructured, conversational text with variable length entries ranging from short phrases to lengthy narrative passages, with diverse linguistic styles reflecting organic user-generated content.

3.7.17: APIS USED:

Web Audio API

The Web Audio API is a high-level JavaScript API provided by modern web browsers for processing and generating audio on web applications. It allows developers to control audio playback, manipulate sound, analyze audio data, and create interactive audio experiences directly within a browser without requiring external plugins.

HTML5 Speech Synthesis API

The HTML5 Speech Synthesis API is part of the Web Speech API that enables web applications to convert text into spoken audio, a process commonly known as Text-to-Speech (TTS). It allows browsers to read text aloud using built-in speech engines and various available voices.

Table 1: Backend APIS

Module	Method	Endpoint URL	Description / Scope
Authentication (routes/auth.py)	POST	/api/signup	Registers a profile, hashes password, generates tags.
	POST	/api/login	Validates hashed credentials to authorize login session.
	POST	/api/login_with_google	Single Sign-On federation check & profile match.
Journaling System (routes/journal.py)	POST	/api/journal/create	Extracts AI metrics, encrypts via Fernet, saves to database.
	GET	/api/journal/<email>	Fetches entry history, decrypting fields dynamically in RAM.
Dashboard & Streaks (routes/dashboard.py)	GET	/api/dashboard/checkin/status/<email>	Validates active date stamps to issue dynamic UI action gates.
	POST	/api/dashboard/checkin/confirm	Commits completion logs and increments active daily streak.
User Settings (routes/settings.py)	GET	/api/settings/<email>	Fetches profile options, themes, and notification toggles.
	PUT	/api/settings/update	Updates core profile configuration metadata fields.
	PUT	/api/settings/change-password	Safely matches and updates user security password.
	DELETE	/api/settings/delete	Cascading profile and encrypted record data purge.

3.8 VERIFICATION AND VALIDATION

3.8.1 VERIFICATION

Technical verification was conducted rigorously using modular framework checks and route execution analysis:

- **API Route Unit Verification:** Isolated blueprints—including user authentication modules, journaling engines, and dashboard state management systems—were sequentially verified utilizing Thunder Client API suites. Individual endpoints were tested to confirm exact structural compliance and HTTP status delivery (e.g., matching payload formats with expected 201 Created and 200 OK statuses).
- **Relative Path Portability Verification:** Initially, the machine learning models were loaded using absolute file paths tied to a single, specific computer (e.g., pointing directly to a specific user folder). This would cause the server to crash if run on any other machine. We fixed this by moving the model binary files (.pkl) directly into the project's root directory under an ml_models/ folder and changing the code to look for them there. This ensures that the fully trained ML model is completely portable and cross-platform so it can now be loaded and run seamlessly on any developer's laptop or evaluation computer without path configuration errors or system crashes.
- **Cryptographic Data-Type Matching Verification:** To ensure complete user data privacy, the system implements field-level symmetric Fernet encryption within the journal routing module. We verified the code structure to ensure data formats transition smoothly without mismatch errors. When a user creates a journal entry, the plain text is explicitly encoded into raw computer data (bytes) for encryption before being saved as secure, unreadable text in MongoDB Atlas. When retrieving history, the server pulls the locked text, decrypts it, and decodes it back into readable text. This process happens strictly within the server's temporary short-term memory (RAM) for a split second while sending the data to the user's screen, ensuring the data is completely protected at rest in the cloud database.

3.8.2 VALIDATION

Functional validation was performed to establish consistency between client interface views, data layers, and analytical outputs:

1. Machine Learning Alignment Validation (AI Consistency Check)

We needed to make absolutely sure that the AI model changes its behavior when we moved it from our teammate's testing notebook into your real Flask web server. To test this, we took the exact same paragraph of text (the one about having a stressful 5 days with heavy workloads) and ran it through both systems:

First, inside your friend's separate Python training notebook.

Second, through a real live request on your backend server.

Because both setups gave back the exact same stress number, we successfully validated that our AI pipeline works perfectly in the real app and makes the exact same smart predictions as it did in the lab.

2. Dynamic Logical Gate Validation (The Cheat-Proof Streak Check)

We tested the app to make sure users can't "cheat" their daily streak scores.

We built a digital security gate based on today's date (YYYY-MM-DD). When a user opens their dashboard, the Flask server instantly checks the database to see if that user actually wrote a journal entry today:

If they haven't written a journal yet: The server sends a `LOCKED` signal, which completely freezes the daily check-in button on the frontend screen.

If they have written a journal: The gate opens, and they are allowed to click the button and increase their streak counter.

CHAPTER 4: EPILOGUE

4.1 TASKS COMPLETED

- Successfully designed the login and sign up pages with proper connection to the database.
- Added user profile, settings and designed a proper interactive dashboard.
- Create journal, analytics, breathing, meditation pages and a proper help section including information about our project.
- Developed backend logic for all successfully created pages.
- Collected datasets and implemented text preprocessing techniques such as tokenization and cleaning of the datasets
- Successfully integrated machine learning–based sentiment analysis to generate sentiment scores from user text inputs.

4.2. TASKS REMAINING

- Design of chatbot interface and its backend logic
- Final integration of the system.
- Backend logic for the charts and graphs section.

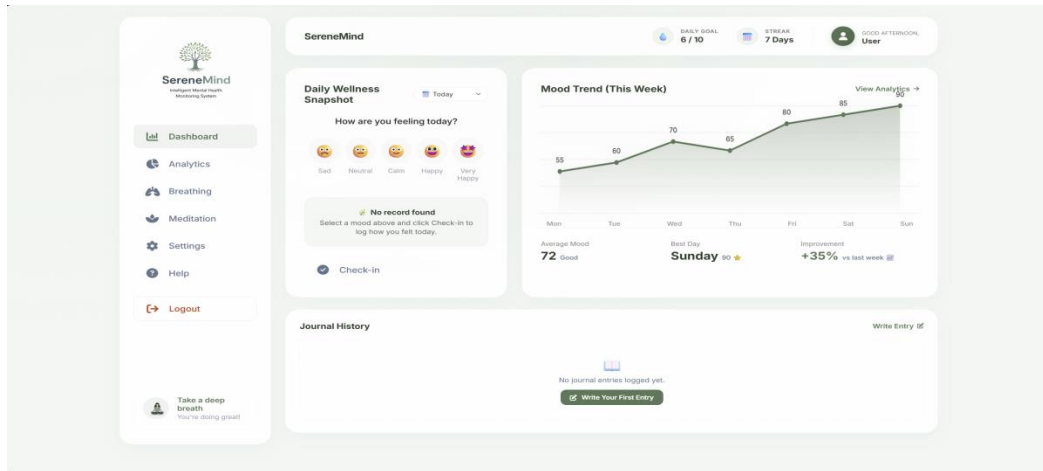
REFERENCES

- [1] K. Saha et al., “Stress Detection from Social Media Using Machine Learning,” ACM, 2019.
- [2] H. Lin et al., “User-Level Psychological Stress Detection from Social Media,” Proceedings of KDD, 2020.
- [3] I. Shah, N. Sheth, D. Shah, V. Sawant, and S. Patil, "Mental Health Assistant Using Machine-Learning," Educational Administration: Theory and Practice, vol. 30, no. 4, pp. 10775–10781, 2024, doi: 10.53555/kuey.v30i4.8346.
- [4] Y. Zhang et al., “Stress Detection from Text Using Machine Learning Models,” IEEE Access, 2021.
- [5] M. A. Khan et al., “Mental Stress Detection Using Machine Learning Techniques: A Review,” IEEE Access, 2022.

BIBLIOGRAPHY

- Admin. (n.d.). *Mental Health App Development Guide in 2026*. KMS Technology. Retrieved January 24, 2026, from <https://kms-technology.com/blog/the-complete-guide-to-mental-health-app-development-in-2026/>
- Jurafsky, & Martin. (n.d.). *Speech and Language Processing* (3rd ed.). Bird et al.
- Kelta, Z. (2024, Aug 8). *Classification in Machine Learning*. Data Camp. <https://www.datacamp.com/blog/classification-machine-learning>
- Admin. (2025, Aug 29). *How Mental Health chatbot Handle stress*. App Solutions. Retrieved 1 17, 2026 <https://theappsolutions.com/blog/development/mental-health-chatbots/>

SCREENSHOTS



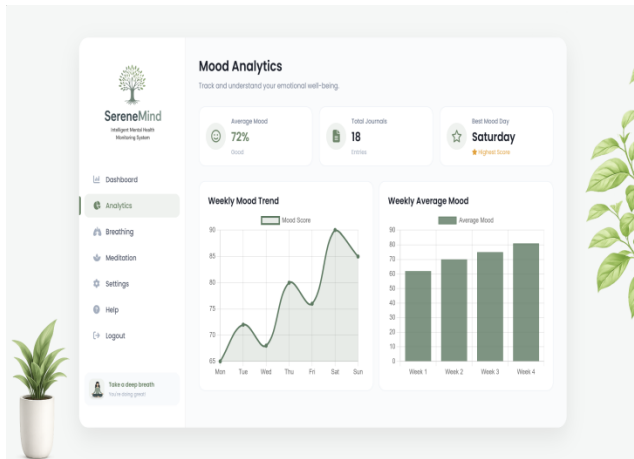
Dashboard

The 'Journal Entry' page is designed for users to document their thoughts and feelings. It features a sidebar with navigation options: Dashboard, Analytics, Breathing, Meditation, Settings, Help, and Logout. The main content area includes a 'New Reflection' form with fields for 'Entry Title...' and 'How was your day?'. A 'Create Entry' button is located at the bottom right of the form. The page also includes a 'Take a deep breath' button and a decorative plant illustration.

Journal page

The 'Help & Support Center' page provides users with resources to get the most out of SereneMind. It includes a sidebar with navigation options: Dashboard, Analytics, Breathing, Meditation, Settings, Help, and Logout. The main content area features a 'Help & Support Center' section with an 'Important Safety & Medical Disclaimer' and a 'Working with SereneMind' section. The 'Frequently Asked Questions' section lists common queries and their answers, such as 'Is my data stored securely and privately?' and 'How does SereneMind generate responses?'. The page also includes a 'Take a deep breath' button and a decorative plant illustration.

Help and support page



Analytics page

Account Settings
Manage your profile and preferences.

Profile Information

Full Name: Abhisha Chughel

Email: abhis@gmail.com

Username: abhis

Security

Current Password: [Masked]

New Password: [Masked]

Confirm Password: [Masked]

Preferences

Email Notifications: [Checked] (Receive updates and reminders via email)

Dark Mode: [Unchecked] (Enable dark mode for this application)

Reminder Frequency: [Checked] (Choose how often to receive check-in prompts)

Three times daily [Checked]

[Delete Account] [Save Changes]

Settings page

Breathing Exercise
Take a moment to center yourself. Follow the guide to sync your breathing and release tension.

Select Session Duration: 1 Minute

Ready 01:00

[Start] [Reset]

Breathing Exercise

Meditation

Session Duration: 3 Minutes

Morning Serenity
Begin your morning with a peaceful practice designed to gently ground your thoughts.

Evening Wind
End your day with a calming practice designed to release stress.

Deep Mind Scan
Explore your mind with a guided practice designed to help you understand your thoughts.

Soothing Mantras
Use these phrases to calm your mind and bring a sense of peace to your practice.

03:00

[Play] [Stop]

Meditation page

```
localhost:27017 > serenemind_db > journals
```

```

{
  "_id": ObjectId("6a2911637198d46af8d57f21"),
  "email": "ayush@gmail.com",
  "title": "gAAAAABqKRFj7WmFzj1Zt8EaYf2jK14Wd8Dj27FNxAu_D69gZJez061MZvzcr0FzvS0LUS...",
  "content": "gAAAAABqKRFj7WmFzj1Zt8EaYf2jK14Wd8Dj27FNxAu_D69gZJez061MZvzcr0FzvS0LUS...",
  "ai_mood": "low",
  "date": "2026-06-10",
  "time_of_creation": 2026-06-10T07:25:23.667+00:00
}

```

```
localhost:27017 > serenemind_db > users
```

```

{
  "_id": ObjectId("6a2911637198d46af8d57f21"),
  "name": "ayush",
  "email": "ayush@gmail.com",
  "password": "ayush123456789",
  "dark_mode": null,
  "email_notifications": null,
  "username": "ayush_gpt"
}

```

MongoDB Implementation